

HCCJP 第76回勉強会

オンプレのサーバー、壊します。

直すのはAIです。

Azure Arc × AIエージェントによる無人障害復旧

2026年8月14日（金） 14:00～15:30

ハイブリッドクラウド研究会

自己紹介

胡田 昌彦（えびすだ まさひこ）

- 日本ビジネスシステムズ株式会社
- Microsoft MVP for Cloud and Datacenter Management / Microsoft Azure
- HCCJP 主幹事
- 最近では **AIエージェント（Claude Code）** に開発・運用のほぼすべてを任せる 生活
- YouTube: 「えびすだまさひこ」で検索 

本日のアジェンダ

時刻	セッション	スピーカー
14:00	オープニング	胡田
14:05	オンプレのサーバー、壊します。直すのはAIです。	胡田
14:50	Q&A	胡田
15:00	Microsoft "Adaptive Cloud" Updates	高添 氏
15:20	Q&A	高添 氏
15:25	クロージング	胡田

今日やること

1. 目の前で、オンプレのサーバーを壊します
2. AIに「監視が赤い。調べて直して」とだけ伝えます
3. 緑に戻るまで、そのままお見せします

壊し方は、皆さんにチャットで決めていただきます。
AIには、何が起きたか一切教えません。

ルール（先に宣言しておきます）

- 🎲 障害はチャットで選んでもらう — 私は当日まで何を壊すか決めません
- 😬 AIに与える情報は一言だけ — 「監視が赤い。原因を調べて直して」
- 🚫 私はサーバーにログインしません — 全部 Azure Arc 越し
- 😊 失敗したらそのまま見せます — うまくいかない様子も含めて実態です

今日の舞台：第74回で建てたラボ

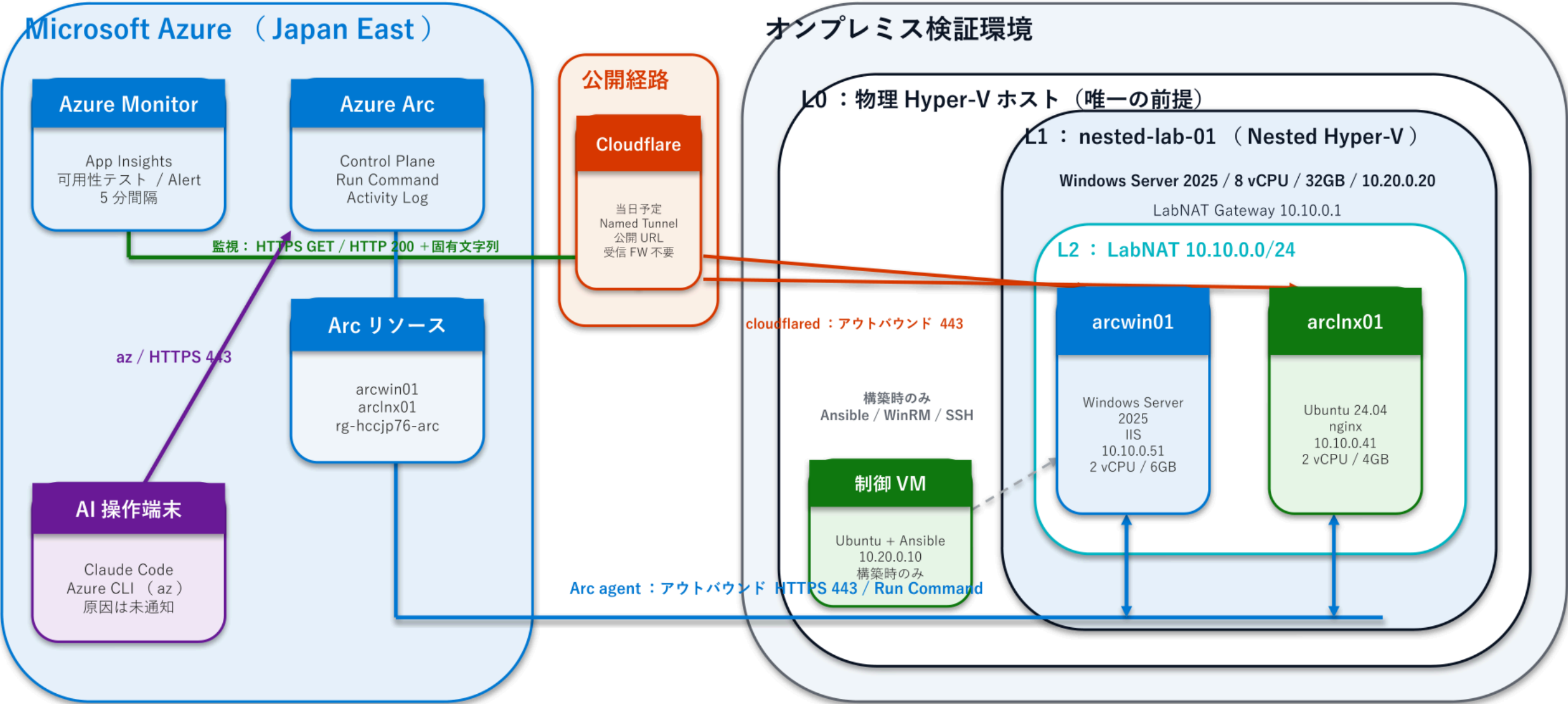
- 第74回「コードで建てる検証環境」の **Nested Hyper-V ラボ**（コードで再構築できる）
- L2 の 2台を Azure Arc に接続済み・どちらも **動いている Web サイト**を持っている

VM	OS / Web	IP	Azure Arc
arcwin01	Windows Server 2025 / IIS	10.10.0.51	Connected (rg-hccjp76-arc / japaneast)
arclnx01	Ubuntu 24.04 / nginx	10.10.0.41	Connected (同上)

【画面】 Azure ポータルの Machines 一覧 → 2台とも Connected を見せる

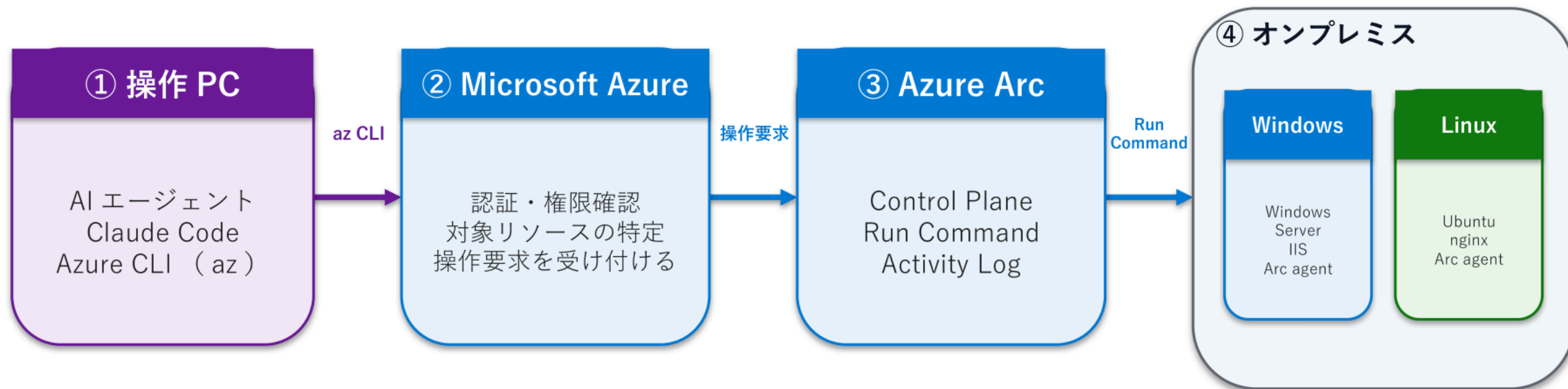
Nested Hyper-V ラボの物理構成

Azure × Cloudflare / 実設定 + 当日予定



青 = Arc 管理経路 緑 = 監視 HTTP 橙 = Cloudflare Tunnel 灰点線 = ラボ構築経路 (ライブ復旧では使用しない)

AI はオンプレミスへ直接接続しない。Azure のコントロールプレーンを経由する。



オンプレ側から Azure へ接続を確立

Connected Machine agent → アウトバウンド HTTPS 443 → 受信ポート開放なし

AI の居場所は操作 PC。Azure Arc が、Azure 上の操作要求をオンプレミスの Windows / Linux へ届ける。

なぜ Azure Arc なのか

オンプレは無くならない

レイテンシ・規制・既存資産・コスト。無くならない前提で考える。

無くせるのは「そこまで行く手間」のほう

障害連絡 → VPN 接続 → 踏み台にログイン → RDP → ようやく調査開始

- 深夜・外出先・移動中.....この経路そのものがボトルネック
- Arc は「サーバーを Azure に移す」話ではなく、**運用の入口を Azure に寄せる話**

繋ぐ仕組み（3分でおさらい）

```
# エージェントを入れて、Azureに繋ぐ。これだけ
azcmagent connect --resource-group rg-hccjp76-arc \
  --tenant-id <tenant> --subscription-id <sub> --location japaneast
```

- 通信はアウトバウンド **443** のみ／インバウンド開放は一切不要
- Arc Gateway でエンドポイントを集約できる
- 繋がると、オンプレのサーバーが **Azure**リソースとして見える

【画面】ポータルで arcwin01 のリソースブレードを開く

Arc越しに使う手段（今日使うのはここ）

やりたいこと	手段
任意コマンドを流す	Run Command ← 今日の主役
対話的に入る	SSH over Arc
構成を宣言して強制する	Machine Configuration
Webの死活を監視する	Azure Monitor / Application Insights
OSのログ・メトリクスを見る	Azure Monitor Agent (AMA)
パッチを当てる	Azure Update Manager

入口が全部 **az** に統一されている = AIエージェントがそのまま手を持てる

AIの手をどう作るか

- MCPではなく「CLI+スキル」で組んでいる（うちの流儀）
 - `az` は枯れている・認証も権限もAzure側で完結する・ログが Activity Log に残る
- AIに教えるのは「正しい叩き方」— 手順をスキルとして文書化しておく
- **渡す権限は絞る** — 触れるサブスクリプション／リソースグループを限定する
- 実際、この環境の準備（Webページ配布まで）も **すべて Run Command 経由** でやった
 - SSHもRDPも使っていない。準備と本番で経路が同じ

監視は「専用ダッシュボード」を作らない

- Application Insights の **Standard 可用性テスト** (5分間隔)
- HTTP 200 + **ページ固有の文字列**を検証 (プロキシのエラーページを成功と誤認しないため)
- オンプレ側にパブリックIPもインバウンド開放も持たせない → **Cloudflare Tunnel** で公開
- ⚠ Arc の Connected Machine heartbeat は**Web障害の信号にならない**
(`Disconnected` 判定まで通常15~30分)

【画面】 Azure Monitor の「可用性」画面 (今は 100% / 緑)

事前検証：どれくらいで赤くなるか（8/5 実測）

イベント	Linux / nginx	Windows / IIS
サービス停止（Run Command）	17:51:03	17:51:46
可用性サンプルが 0%	17:55	17:55
Azure Monitor アラート発火	17:57:15	17:57:13
停止 → 発火	6分12秒	5分27秒
復旧（Run Command） → 100%	18:00	18:00

赤くなるまで数分かかります。その間は雑談タイムです 🍲

それでは、壊します 🛠️

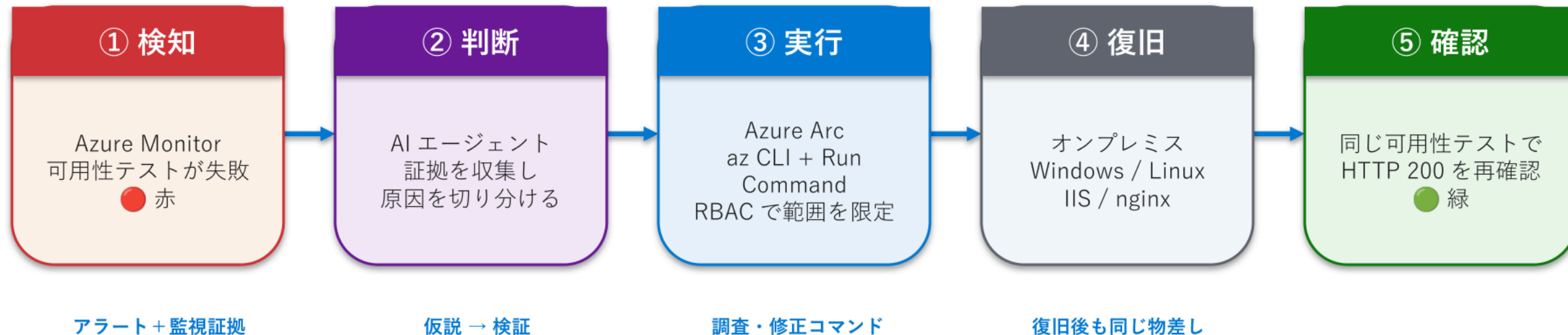
チャットで番号を選んでください

障害シナリオ（当日提示する選択肢）

#	何を壊すか	OS
1	Webサービスを止める	Linux / Windows
2	設定ファイルを壊して起動不能にする	Linux
3	ファイアウォールで塞ぐ（サービスは正常なのに繋がらない）	Windows
4	ログ肥大でディスクを満杯にする	Linux
5	サービスアカウントのパスワードを変える	Windows

#3 と #5 は「症状と原因が一致しない」タイプ。AIが"考えている"のが一番よく見えます。

障害は視聴者が選ぶ。AI は対象・症状・原因を知らない。



人はサーバーへ直接ログインしない

VPN なし | 踏み台なし | インバウンド開放なし | Activity Log + AI 実行ログで追跡

ポイント：成功そのものより、証拠を取ってから動くか／失敗時に次の仮説へ進めるかを見る

● 監視が赤くなりました

AIに渡す言葉は、これだけ

「監視が赤い。原因を調べて直して」

【画面】 ここからAIのターミナルを大きく映す。以降しばらくスライドに戻らない

ライブ中に注目してほしいところ 🙄

- AIが **どこから調べ始めるか** (対象の特定 → 症状の確認 → 原因の切り分け)
- **推測で直そうとしないか** (証拠を取ってから動いているか)
- 一度で直らなかったとき、**次に何を試すか**
- そして —— **私は一度もサーバーにログインしていない**

● 復旧しました（といいですね）

【画面】可用性テストが100%に戻るところを見せる

種明かし：AIは何を叩いたのか

- 実行された `az` / Run Command を **全部**並べて振り返る
- 流れを色分けして見る：**調査** → **仮説** → **検証** → **修正** → **再確認**
- 直らなかった試行も消さずに見せる（そこが一番情報量が多い）
- 同じものが **Azure Activity Log** にも残っている
→ 「誰が」「いつ」「どのリソースに」「何をしたか」が Azure 側の記録として残る

【画面】Activity Log を開いて Run Command の記録を見せる

.....で、怖くないですか？ 🙄

- Run Command は **任意のコマンドを実行できる** = 最強かつ最恐
- AIが暴走したら？ 誤った対象に流したら？
- でも —— **人間がやっても同じリスクはある**
- 違うのは、**記録が残るかどうか**と、**範囲を先に決められるかどうか**

「できる」と「やらせていい」の線引き

論点	現実的な落とし所
権限	スコープを切る／カスタムロール／まず読み取り専用の世界を作る
対象	タグやリソースグループで、AIが触れる範囲を物理的に限定
実行	破壊的操作は 承認ゲート を挟む（最後は人が押す）
記録	Activity Log + AIの実行ログ。 人間の手作業より追跡しやすい
練習	まず検証環境で流す（今日のラボがまさにそれ）

「AIに全部やらせる／やらせない」の二択にしないこと。

まとめ

- **運用する場所**が、物理的な「そこ」から外れた (Azure Arc)
- そこにAIが入ると、**手を動かす人**が要らなくなる場面が出てくる
- では、インフラエンジニアは何をする人になるのか
 - 環境を**設計**する人
 - AIに渡す**権限と境界**を決める人
 - 起きたことを**説明**できる人
- 今日の実験がうまくいっても、いかなくても、**線引きの議論は先に始められます**

Q&A

チャット・コメントからどうぞ 🙋

次のセッション

Microsoft "Adaptive Cloud" Updates

高添 修 氏（日本マイクロソフト株式会社）

Azure Local / Azure Arc / Windows Server の最新動向

ご参加ありがとうございました！

- HCCJP は **Azure × ハイブリッドクラウド × 生成AI** のコミュニティです
- 企業・個人問わずオープンに参加できます
- **登壇者・事例共有 募集中！** 「うちのこの話、してみたい」をぜひ 🙌
- Connpass: <https://hybridcloud.connpass.com/>

次回：第77回（2026年9月11日（金）14:00～）